Python Strings: Index, Slice & Encoding — Cheat Sheet

Nov 5, 2026 · @Aavisek Choudhury

Mental model

A Python `str` is an **immutable sequence of Unicode code points**: you can read any character by position, but you can never change one in place.

```
s = "PYTHON"

  +---+---+---+---+---+---+
  | P | Y | T | H | O | N |
  +---+---+---+---+---+---+
  0   1   2   3   4   5   6     <- boundaries from the left (6 = len(s))
 -6  -5  -4  -3  -2  -1         <- same boundaries from the right
```

- **Index** `s[i]` returns the one character just right of line `i`, as a 1-char `str`.
- **Slice** `s[a:b]` returns a new `str` holding everything between line `a` and line `b`.
- **Encode** `s.encode()` turns the `str` into `bytes`; `.decode()` turns the bytes back into a `str`.

Indexing

`s[i]` returns a 1-character string; a negative `i` counts from the end, and anything outside `-len(s) <= i < len(s)` raises `IndexError`.

```
s = "PYTHON"
s[0]         # 'P'   first char
s[5]         # 'N'   last char
s[-1]        # 'N'   last char (the idiomatic way)
s[-6]        # 'P'   first char, counted from the end
s[6]         # IndexError: string index out of range
s[0] = "J"   # TypeError: 'str' object does not support item assignment
```

Strings are immutable, so "changing" a character means building a new string: `"J" + s[1:]` gives `'JYTHON'`, and `s.replace("P", "J", 1)` does the same.

Finding an index

Call	Returns	When not found
`s.find(sub, start, end)`	lowest index of `sub`	`-1`
`s.rfind(sub)`	highest index of `sub`	`-1`
`s.index(sub)`	lowest index of `sub`	raises `ValueError`
`s.rindex(sub)`	highest index of `sub`	raises `ValueError`
`s.count(sub)`	number of non-overlapping matches	`0`
`sub in s`	`True` / `False`	`False`
`s.startswith(p)` / `s.endswith(p)`	`True` / `False` (accepts a tuple of prefixes)	`False`

```
t = "banana"
t.find("an")      # 1
t.rfind("an")     # 3
t.find("a", 2)    # 3    start searching at index 2
t.find("x")       # -1   not found, no exception
t.index("x")      # ValueError: substring not found
t.count("an")     # 2
t.count("ana")    # 1    non-overlapping: the 2nd "ana" shares letters with
the 1st

for i, ch in enumerate(t):   # index + char together
    print(i, ch)
```

Slicing

`s[start:stop:step]` returns a new string from `start` up to, but not including, `stop`; every part is optional, and out-of-range bounds are clipped instead of raising an error.

```
s = "PYTHON"
s[2:5]       # 'THO'     indexes 2,3,4  (stop is excluded)
s[:3]        # 'PYT'     first 3
s[3:]        # 'HON'     from index 3 to the end
s[-3:]       # 'HON'     last 3
s[:-1]       # 'PYTHO'   everything except the last char
s[1:-1]      # 'YTHO'    trim one char from both ends
s[::2]       # 'PTO'     every 2nd char
s[1::2]      # 'YHN'     every 2nd char, starting at index 1
s[::-1]      # 'NOHTYP'  reversed
s[4:1:-1]    # 'OHT'     indexes 4,3,2 (walks left; stop still excluded)
s[2:100]     # 'THON'    stop past the end is clipped, no error
s[5:2]       # ''        start is past stop with a positive step -> empty
s[::0]       # ValueError: slice step cannot be zero
```

Part	If omitted	Notes
`start`	`0` (step > 0) or the last char (step < 0)	inclusive
`stop`	`len(s)` (step > 0) or "before the first char" (step < 0)	exclusive, so it never appears in the result
`step`	`1`	negative walks right to left; `0` raises `ValueError`

- `s[:n] + s[n:] == s` for any `n`: the two halves never overlap and never drop a character.
- `len(s[a:b]) == b - a` whenever `0 <= a <= b <= len(s)`.
- A slice never raises `IndexError`; only a single index does.
- A slice is an object you can name and reuse: `sl = slice(2, 5)` then `s[sl]` gives `'THO'`, and `sl.indices(len(s))` gives `(2, 5, 1)`.

Slice recipes

Thirteen one-liners cover most real-world slicing; every result below is for `s = "Hello, World"`.

Goal	Code	Result
First 5 chars	`s[:5]`	`'Hello'`
Last 5 chars	`s[-5:]`	`'World'`

Goal	Code	Result
Drop first and last char	`s[1:-1]`	`'ello, Worl'`
Reverse	`s[::-1]`	`'dlroW ,olleH'`
Every other char	`s[::2]`	`'Hlo ol'`
Middle chunk (index 3 to 7)	`s[3:8]`	`'lo, W'`
Split in half	`s[:len(s)//2]`, `s[len(s)//2:]`	`'Hello,'`, `' World'`
Safe first char (no `IndexError` on `""`)	`s[:1]`	`'H'` (`''` if empty)
Insert `!` at index 5	`s[:5] + "!" + s[5:]`	`'Hello!, World'`
Replace the char at index `i`	`s[:i] + new + s[i+1:]`	new `str`
Rotate left by 2	`s[2:] + s[:2]`	`'llo, WorldHe'`
Split into chunks of 4	`[s[i:i+4] for i in range(0, len(s), 4)]`	`['Hell', 'o, W', 'orld']`
Drop a known prefix / suffix (3.9+)	`s.removeprefix("Hello, ")`, `s.removesuffix("World")`	`'World'`, `'Hello, '`

Palindrome check: `t == t[::-1]`.

Encoding basics

`str` is text (code points) and `bytes` is raw data (values 0-255); `.encode()` turns a `str` into `bytes`, `.decode()` turns `bytes` back, and both sides must use the same encoding. UTF-8 is the default in Python 3.

```
str  --- .encode("utf-8") --->  bytes
str  <--- .decode("utf-8") ---  bytes
```

```
s = "café €"
b = s.encode("utf-8")    # b'caf\xc3\xa9 \xe2\x82\xac'
b.decode("utf-8")        # 'café €'
len(s)                   # 6   code points
len(b)                   # 9   bytes (é takes 2, € takes 3)
s.encode()               # same as encode("utf-8"); decode() defaults to UTF-
8 too
```

Common encodings

Encoding	Bytes per char	Covers	Use it for
`utf-8`	1 to 4	all of Unicode; ASCII-compatible	files, web, APIs: the default choice
`ascii`	1	128 chars (U+0000 to U+007F)	strict English-only data
`latin-1`	1	256 chars (U+0000 to U+00FF); every byte value decodes	legacy Western European data
`cp1252`	1	Windows Western European	legacy Windows text files
`utf-16`	2 or 4, plus a BOM	all of Unicode	Windows/Java internals; use `utf-16-le` / `utf-16-be` for no BOM
`utf-32`	4, plus a BOM	all of Unicode	fixed width, rarely used

UTF-8 size rule: 1 byte up to U+007F, 2 bytes up to U+07FF, 3 bytes up to U+FFFF, 4 bytes above that.

```
"A".encode("utf-8")        # b'A'                  1 byte
"é".encode("utf-8")        # b'\xc3\xa9'           2 bytes
"€".encode("utf-8")        # b'\xe2\x82\xac'       3 bytes
"😀".encode("utf-8")       # b'\xf0\x9f\x98\x80'   4 bytes
"é".encode("latin-1")      # b'\xe9'               1 byte
"€".encode("latin-1")      # UnicodeEncodeError    no € in Latin-1
"A".encode("utf-16")       # b'\xff\xfeA\x00'      BOM + 2 bytes (little-
endian machine)
"A".encode("utf-16-le")    # b'A\x00'              no BOM
```

Encoding errors and handlers

Both `.encode()` and `.decode()` raise by default (`errors="strict"`); pass `errors=` to pick a handler when the text and the encoding do not match.

- `UnicodeEncodeError`: a character in the `str` does not exist in the target encoding.
- `UnicodeDecodeError`: the bytes are not valid in the encoding you named.

Unless a row shows a decode, results are for `"café".encode("ascii", errors=...)`:

`errors=`	Works on	Effect	Result
`"strict"` (default)	both	raise the error	`UnicodeEncodeError`
`"ignore"`	both	silently drop the bad part	`b'caf'`
`"replace"`	both	`?` when encoding, `&#65533;` (the &#65533; mark) when decoding	`b'caf?'`
`"backslashreplace"`	both	insert a Python escape	`b'caf\\xe9'`
`"xmlcharrefreplace"`	encode only	insert an HTML/XML entity	`b'café'`
`"namereplace"`	encode only	insert the Unicode name	`b'caf\\N{LATIN SMALL LETTER E WITH ACUTE}'`
`"surrogateescape"`	both	smuggle undecodable bytes through as lone surrogates, so they round-trip	`b'caf\xe9'` decodes to `'caf\udce9'`

```
bad = b"caf\xe9"                          # Latin-1 bytes, not valid UTF-8
bad.decode("utf-8")                       # UnicodeDecodeError: ... can't decode
byte 0xe9 in position 3
bad.decode("utf-8", "replace")            # 'caf&#65533;'
bad.decode("utf-8", "ignore")             # 'caf'
bad.decode("latin-1")                     # 'café'   the right encoding beats
any handler
```

Reading garbled text

What you see	Likely cause	Fix
`cafÃ©` instead of `café`	UTF-8 bytes were decoded as Latin-1 / cp1252	`s.encode("latin-1").decode("utf-8")`
`café` marks in the text	decoded with `errors="replace"`, so the data is already lost	re-read the original bytes with the correct encoding
`` at the start of the first line	file has a UTF-8 BOM (common from Excel)	open with `encoding="utf-8-sig"`
`UnicodeDecodeError` on a UTF-8 read	file is not UTF-8	try `cp1252` or `latin-1`, or detect with `charset-normalizer`
`“hi”` turns into odd symbols	Windows "smart quotes" read as Latin-1	decode as `cp1252`

Unicode helpers and file I/O

`ord()` and `chr()` convert between a character and its code point, `unicodedata` tames lookalike characters that make `len()` and `==` surprising, and every file open needs an explicit encoding.

Code points and escapes

```
ord("A")           # 65
ord("€")           # 8364   (0x20AC)
chr(8364)          # '€'
"€"           # '€'    16-bit escape
"\U0001F600"       # '😀'    32-bit escape
"\N{EURO SIGN}"    # '€'    escape by Unicode name
```

Normalization and case

```
import unicodedata as ud
a = "é"                      # 'é' as ONE code point  (NFC form)
b = "é"                     # 'e' + combining accent  (NFD form), looks
identical
a == b                           # False
len(a), len(b)                   # (1, 2)
ud.normalize("NFC", b) == a      # True   normalize before comparing or
hashing
ud.name("é")                    # 'LATIN SMALL LETTER E WITH ACUTE'
"Straße".casefold()             # 'strasse'   use casefold(), not lower(),
for caseless matching
```

bytes behave differently from str

```
b = "café".encode("utf-8")       # b'caf\xc3\xa9'
b[0]                             # 99        indexing bytes gives an int
b[0:1]                           # b'c'      slicing bytes gives bytes
b[3:4].decode("utf-8")           # UnicodeDecodeError: sliced through the
middle of é
bytearray(b)                     # mutable copy: ba[0] = 67 works
b.hex()                          # '636166c3a9'
bytes.fromhex("c3a9").decode()   # 'é'
```

Files

```
with open("data.txt", "w", encoding="utf-8") as f:   # text mode: you write
str
    f.write("café")
with open("data.txt", "rb") as f:                    # binary mode: you get
bytes
    f.read()                                         # b'caf\xc3\xa9'
```

Always pass `encoding=` to `open()`: without it, Python uses the platform's default, which is often `cp1252` on Windows and can differ from the machine that wrote the file.

Gotchas and quick recall

Most string bugs come from ten mistakes; the left column is what people write, the right column is what to write instead.

Mistake	What goes wrong	Do this instead
`if s.find("x"):`	`find` returns `-1` (truthy) when missing and `0` (falsy) at the start	`if "x" in s:` or `if s.find("x") != -1:`
`s[len(s)]` or `s[-0]`	index `len(s)` is out of range; `-0` is just `0`	`s[-1]` for the last char
Treating `stop` as inclusive	`s[2:5]` has 3 chars, not 4	length is `stop - start`
`s[0] = "J"` or `s += x` in a loop	strings are immutable; the loop rebuilds the string every pass	slice and rebuild, or `"".join(parts)`
`str(b"hi")`	returns `"b'hi'"`, the repr, not `"hi"`	`b"hi".decode("utf-8")`
`"a" == b"a"`	always `False`: `str` never equals `bytes`	encode or decode first, then compare
`len(s)` as the visible length	counts code points: `"👨‍👩‍👧"` is 5, `"é"` is 2	`unicodedata.normalize("NFC", s)`, or the `regex` package's `\X` for graphemes
`s[::-1]` on accented text	`"é"[::-1]` puts the accent before the letter	normalize to NFC before reversing
`open(path)` with no encoding	platform default decides, so the same code reads differently on two machines	`open(path, encoding="utf-8")`
`errors="ignore"` to silence an error	silently deletes data	fix the encoding; use `replace` only when loss is acceptable

Remember it in six lines

- **Index:** `s[0]` first, `s[-1]` last, outside the range raises `IndexError`.
- **Slice:** `s[start:stop:step]`, stop is never included, nothing raises, `s[::-1]` reverses.
- **Find:** `find` gives `-1`, `index` raises `ValueError`.
- **Immutable:** build a new string; never assign into one.

- **Encode to ship, decode to show:** `str.encode("utf-8")` makes bytes, `bytes.decode("utf-8")` makes text, and both ends must name the same encoding.
- **Sizes differ:** `len(str)` counts code points, `len(bytes)` counts bytes, and UTF-8 spends 1 to 4 bytes per code point.

Sample program: Should You Go for a Walk?

With `is_sunny = True` and `have_umbrella = False`, three of the four checks print `True`: `not` flips a boolean, `and` needs both sides `True`, and `or` needs at least one.

```
is_sunny = True
have_umbrella = False

# 1. Is it not sunny today?
print("1. Is it not sunny today?", not is_sunny)

# 2. Do you not have an umbrella?
print("2. Do you not have an umbrella?", not have_umbrella)

# 3. Sunny AND you don't need an umbrella (read as: you don't have one)
print("3. Walk if it's sunny and you don't need an umbrella?", is_sunny and
not have_umbrella)

# 4. Sunny OR you have an umbrella
print("4. Walk if it's sunny or you have an umbrella?", is_sunny or
have_umbrella)
```

Output:

```
1. Is it not sunny today? False
2. Do you not have an umbrella? True
3. Walk if it's sunny and you don't need an umbrella? True
4. Walk if it's sunny or you have an umbrella? True
```

Check 3 reads "don't need an umbrella" as `not have_umbrella`, since that is the only umbrella variable given.